Technical & Code Guide

Adaptive AI Phishing Detection System

Every file. Every line. Every concept. Explained like you are 5.

Final Year Project · Computer Science · 2024/2025

#	Section	What You Will Understand
1	How Python Actually Works	What Python is and why it runs your project
2	Your Folder & File Structure	What every file does and why it exists
3	requirements.txt	What libraries are and how pip installs them
4	generate_dataset.py	How the training data is created line by line
5	train_model.py	How the AI brain is built and trained
6	app.py — Part 1: Setup	How the app starts, loads the model, and styles itself
7	app.py — Part 2: The Sidebar	How the navigation and system status panel works
8	app.py — Part 3: Chat Logic	How a message goes from input to verdict on screen
9	app.py — Part 4: Other Pages	Statistics, History, Settings, and About pages

10	The Full Data Journey	One complete walkthrough from typing to result
11	Common Errors & What They Mean	What goes wrong and how to read error messages

1 How Python Actually Works

Before we touch any code

What Is Python?

Python is a programming language. A programming language is just a way to give instructions to a computer. Like how you give instructions to a person in English, you give instructions to a computer in Python.

> ■ **Simple Analogy:** Imagine a robot that can do any task — but it only speaks one language. Python is that language. Every file ending in **.py** is a page of instructions written in that language. When you run it, the robot reads and follows those instructions from top to bottom, one line at a time.

What Happens When You Type 'python train_model.py'?

When you type that command in the terminal and press Enter, here is exactly what happens:

Step	What Happens	Think of it as...
1	Python opens the file train_model.py	You open a recipe book to a specific page
2	It reads the file from the very first line	You start reading the recipe from step 1
3	It executes each line one by one, top to bottom	You follow each step of the recipe in order
4	If it hits an error, it stops and shows you the problem	You realize you are missing an ingredient
5	When it reaches the last line, it finishes	The dish is done — the program is done

What Is The Terminal?

The terminal (also called Command Prompt on Windows) is a text box where you talk directly to your computer using typed commands. It has no buttons or menus — just a blinking cursor waiting for you to type.

> ■ **Think of the terminal like texting your computer.** Instead of clicking buttons, you type exactly what you want the computer to do. 'pip install streamlit' means 'hey computer, go download and install streamlit for me.'

2 Your Folder & File Structure

What every file does and why it exists

Your project folder called **Adaptive-Phishing-System** contains several files and sub-folders. Each one has a specific job. Nothing is there by accident.

File / Folder	Type	Its Job — What It Does
requirements.txt	Text file	Shopping list of Python libraries to install
generate_dataset.py	Python file	Creates your training data (280 messages with labels)
train_model.py	Python file	Uses training data to build and save the AI brain
app.py	Python file	The full website/interface — runs the whole system
data/	Folder	Stores all data files
data/dataset.csv	CSV file	The 280 labeled messages the AI learned from
data/feedback.csv	CSV file	Corrections users give — created automatically
models/	Folder	Stores the trained AI brain files
models/phishing_model.pkl	PKL file	The saved trained AI brain — Random Forest model
models/vectorizer.pkl	PKL file	The saved TF-IDF translator
models/metrics.json	JSON file	The accuracy scores saved after training
logs/	Folder	Stores scan history
logs/scan_log.csv	CSV file	Log of every message ever scanned — auto-created

What Is a .pkl File?

PKL stands for Pickle. In Python, 'pickling' means saving a complex object (like a trained AI model) into a file so you can load it again later without having to rebuild it from scratch every single time.

> ■ **Think of it like saving a game.** When you train the AI model, it takes time and effort — like building a save point in a video game. The .pkl file IS that save point. When the app starts, it loads the .pkl file and continues from exactly where training left off — instantly.

What Is a .csv File?

CSV stands for Comma Separated Values. It is the simplest type of spreadsheet. Each row is one record. Each column is separated by a comma. Your dataset.csv has two columns: **text** (the message) and **label** (1 for phishing, 0 for safe).

3 requirements.txt

The Shopping List

What This File Does

This is the simplest file in your project. It is literally just a list of extra tools (called libraries) that your project needs. Python comes with basic abilities built in — but for advanced things like machine learning and building websites, you need to install extra libraries.

> ■ **Think of libraries like apps on your phone.** Your phone came with a basic camera app already installed. But to edit photos professionally, you download Adobe Lightroom. Libraries are like that — extra power you download and add to Python. The requirements.txt file tells Python exactly which apps to download.

Your requirements.txt — Line by Line:

Library Name	What It Does	Analogy
streamlit	Turns your Python code into a website	The architect who builds the visible house
pandas	Loads and handles your CSV data files	Excel inside Python
numpy	Does fast math with big lists of numbers	A super-fast calculator
scikit-learn	Contains the Random Forest + TF-IDF tools	The actual AI toolkit
joblib	Saves and loads your trained model (pkl)	The save/load function for your AI

The Command That Uses This File:

```
# This one command reads requirements.txt and installs everything in it
pip install -r requirements.txt
# pip = Python's app store
# install = download and set up
# -r = read from a file
# requirements.txt = the file to read from
```

4

generate_dataset.py

Creating The AI's Training Data

What This File Does — The Big Picture

This file creates the training data your AI learns from. Without this file, the AI has nothing to study. It creates 280 messages — some phishing, some legitimate — and saves them to data/dataset.csv.

> ■ **Think of it like a teacher preparing an exam study guide.** The teacher writes 140 examples of wrong answers (phishing) and 140 examples of correct answers (legitimate). The student (AI) studies all 280 examples until it can tell the difference perfectly.

The File — Section by Section:

Line 1-2: Import Tools

```
# Import means 'bring in a tool I need'
import pandas as pd
import random
```

import pandas as pd — Brings in the pandas library and gives it the nickname 'pd' so you do not have to type 'pandas' every time. Pandas is used to create the CSV file at the end.

import random — Brings in Python's built-in random number generator. Used to shuffle the dataset so phishing and legitimate messages are mixed together, not grouped.

The Phishing Messages List

```
# This is a Python LIST — a collection of items inside square brackets []
# Each item is a STRING — text wrapped in quotes
phishing = [
"Your account is locked. Verify at http://bit.ly/xyz",
"URGENT: PayPal suspended. Click: http://paypa1-secure.com",
```

```
"Win $1,000,000. Claim at http://free-prize-winner.tk",
# ... 37 more phishing messages ...
]
```

A list in Python is like a numbered box. You put things inside it using square brackets []. Each item is separated by a comma. This list contains 40 real-looking phishing messages.

The Legitimate Messages List

```
legitimate = [
"Hey, are we still meeting for lunch tomorrow?",
"Your Amazon order has been delivered.",
"Please find attached the quarterly report.",
# ... 37 more safe messages ...
]
```

Building The Combined Dataset

```
# Start with an empty list
data = []

# Loop through every phishing message and add it with label 1
for msg in phishing:
data.append({"text": msg, "label": 1})

# Loop through every legitimate message and add it with label 0
for msg in legitimate:
data.append({"text": msg, "label": 0})
```

> ■ **for msg in phishing:** means 'go through every single message in the phishing list, one by one.' Each time, msg becomes that message.
>
> **data.append({...})** means 'add this item to the data list.' Each item is a dictionary (key-value pair): the text of the message and its label (1 = phishing, 0 = safe).

Saving to CSV

```
# Convert the list into a pandas DataFrame (like a spreadsheet)
df = pd.DataFrame(data + extra)

# Shuffle the rows so phishing and safe are mixed randomly
df = df.sample(frac=1, random_state=42).reset_index(drop=True)
```

```
    # Save the spreadsheet as a CSV file
    df.to_csv('data/dataset.csv', index=False)

    # Print a summary to the terminal so you know it worked
    print(f"Dataset created: {len(df)} samples")
```

5

train_model.py

Building and Training The AI Brain

What This File Does — The Big Picture

This is the most important technical file. It reads your dataset, teaches the Random Forest AI to recognise phishing patterns, tests how good it is, and saves the trained brain to a file. You only need to run this once (or again whenever you want to retrain with new feedback).

Step 1: Importing All The Tools

```
# pandas reads the CSV file
import pandas as pd
# train_test_split divides data into training and testing portions
# cross_val_score tests the model 5 different ways for reliability
from sklearn.model_selection import train_test_split, cross_val_score
# TfidfVectorizer converts text into numbers
from sklearn.feature_extraction.text import TfidfVectorizer
# RandomForestClassifier IS the AI model — 200 decision trees
from sklearn.ensemble import RandomForestClassifier
# These measure how accurate the model is
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix
# joblib saves and loads the trained model
import joblib
```

Step 2: Loading The Dataset

```
df = pd.read_csv(DATA_PATH)
# df stands for DataFrame — like a spreadsheet loaded into memory
# It now has two columns: 'text' and 'label'
# Example of what df looks like:
# text | label
# 'URGENT: Verify your account now...' | 1
# 'Hey, meeting at 3pm tomorrow?' | 0
```

■ **pd.read_csv()** opens the CSV file and loads it into a DataFrame. A DataFrame is like an Excel spreadsheet that lives inside Python's memory. You can filter it, add columns, do math on it — everything.

Step 3: Separating Features From Labels

```
# X = the INPUT — the actual message text
X = vectorizer.fit_transform(df['text'])

# y = the ANSWER — is it phishing (1) or safe (0)?
y = df['label']
```

■ In machine learning, we always separate the **inputs** (called X) from the **answers** (called y).

X = the question (the message text, converted to numbers by TF-IDF)
y = the correct answer (1 for phishing, 0 for safe)

The AI studies thousands of (X, y) pairs to learn the pattern between them.

Step 4: TF-IDF — Converting Text to Numbers

```
# Create the TF-IDF converter
vectorizer = TfidfVectorizer(
ngram_range=(1, 2), # look at single words AND pairs of words
max_features=5000, # only keep the 5000 most important words
sublinear_tf=True # use log math to reduce the effect of repeated words
)

# fit_transform = learn from AND convert the data at the same time
X = vectorizer.fit_transform(df['text'])
# X is now a matrix of numbers — every message is now a row of scores
```

■ **ngram_range=(1,2)** means it looks at individual words ('urgent') AND pairs of words ('verify account', 'click here'). Pairs of words give more context than single words alone.

max_features=5000 means it only tracks the 5000 most useful words. Keeping all words would be wasteful and slow.

After this step, a message like 'URGENT: verify your account' becomes a row of 5000 numbers — most are 0, a few are high scores for 'urgent' and 'verify'.

Step 5: Splitting Into Training and Testing Data

```
X_train, X_test, y_train, y_test = train_test_split(
X, y,
test_size=0.2, # 20% of data goes to testing
random_state=42, # same random split every time (reproducible)
stratify=y # keep same ratio of phishing/safe in both sets
)
```

✂■ **Why split the data?** If you studied from a textbook AND the exam questions were the exact same questions from that textbook — you would get 100% just by memorising. That proves nothing.

The split ensures the AI is tested on messages it has *never seen before*. **80% is used for studying. 20% is locked away for the exam.** The AI never sees the test set during training.

Step 6: Training The Random Forest

```
model = RandomForestClassifier(
n_estimators=200, # create 200 decision trees
max_depth=None, # let trees grow as deep as needed
min_samples_split=2, # split a node if it has at least 2 samples
random_state=42, # same result every run
n_jobs=-1 # use all CPU cores for speed
)

# THIS is the actual training — the AI studies X_train and y_train
```

```
model.fit(X_train, y_train)
```

■ **model.fit(X_train, y_train)** is the single most important line in the file. This is where the AI actually learns.

It shows all 200 decision trees the training messages (X_train) and their correct labels (y_train). Each tree learns to ask questions like: 'Does this message contain the word urgent? Does it have a suspicious URL?' Until it can correctly classify the training examples.

Step 7: Testing and Evaluating

```
# Ask the model to predict labels for the test messages
y_pred = model.predict(X_test)

# Compare predictions to the real labels — calculate scores
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

# Cross-validation: test 5 times on 5 different splits
cv_scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')
```

Step 8: Saving The Model

```
# Save the trained AI brain to a file
joblib.dump(model, 'models/phishing_model.pkl')

# Save the TF-IDF converter — needed to process future messages
joblib.dump(vectorizer, 'models/vectorizer.pkl')

# Save the accuracy scores to a JSON file
with open('models/metrics.json', 'w') as f:
json.dump(metrics, f, indent=2)
```

■■ **Why save the vectorizer too?** When a new message arrives in the app, it must be converted to numbers in EXACTLY the same way as the training data. The vectorizer remembers all 5000 words and their scores. Without loading the same vectorizer, the numbers would mean different things and the model would give wrong answers.

6 app.py — Part 1: Setup & Styling

How The App Starts Up

What app.py Is

app.py is the biggest and most important file. It is the entire web interface — the chat, the sidebar, the pages, the buttons, everything the user sees and touches. When you run 'streamlit run app.py', Streamlit reads this file and builds a website from it.

The Very First Line — Page Configuration

```
# This MUST be the first Streamlit command — sets up the browser tab
st.set_page_config(
page_title="Adaptive AI Phishing Detection System",
page_icon="■■",
layout="wide", # use full browser width
initial_sidebar_state="expanded" # sidebar open by default
)
```

> ■■ If you put any other Streamlit command before this one, the app will crash with an error. This line must always come first — it is like setting up your workspace before you start working.

Loading The Model — @st.cache_resource

```
@st.cache_resource
def load_model():
model = joblib.load("models/phishing_model.pkl")
vectorizer = joblib.load("models/vectorizer.pkl")
return model, vectorizer
```

■ **@st.cache_resource** is a decorator — think of it as a sticky note on the function that says: 'Only run this once. After that, remember the result and use it again without re-loading.'

Without this, the app would reload the AI model from disk every single time the user clicks a button — which would be very slow. With it, the model loads once when the app starts and stays in memory.

Session State — Remembering Things Between Clicks

```
# Session state is like the app's short-term memory
if 'messages' not in st.session_state:
st.session_state.messages = []

if 'active_page' not in st.session_state:
st.session_state.active_page = "Chat"

if 'feedback_count' not in st.session_state:
st.session_state.feedback_count = 0
```

■ **Why is this needed?** Every time you click a button in Streamlit, the ENTIRE Python script reruns from the very top. Without session_state, all your chat messages would disappear on every click.

st.session_state is a special dictionary that survives reruns. Anything stored in it stays alive as long as the browser tab is open. The 'if not in' check ensures we only set the default value once — not on every rerun.

The CSS Styling

The large block of CSS code inside st.markdown() controls all the visual styling — the dark background, the chat bubble colors, the fonts, the sidebar gradient. CSS is a styling language for websites.

```
# st.markdown() lets you inject HTML and CSS directly into the page
st.markdown('''
<style>
.stApp { background-color: #0f1117; } /* dark background */
.user-msg { background: #2d3561; border-radius: 18px; } /* blue bubble */
```

```
    .bot-msg.phishing { border-left: 4px solid #ff4d6d; } /* red left border */
    .bot-msg.safe { border-left: 4px solid #00d4aa; } /* green left border */
    </style>
    ''', unsafe_allow_html=True)
```

7

app.py — Part 2: The Sidebar

Navigation and System Status

Building The Sidebar

```
# Everything inside this block appears in the left sidebar
with st.sidebar:
st.markdown("""...""", unsafe_allow_html=True)

# Create a navigation button for each page
pages = [('■','Chat'), ('■','History'), ('■','Statistics'), ...
for icon, name in pages:
if st.button(f"{icon} {name}", key=f"nav_{name}"):
st.session_state.active_page = name
st.rerun()
```

■ **with st.sidebar:** means 'everything I write inside here goes into the left sidebar panel, not the main page.'

st.button() creates a clickable button. When clicked, it returns True and the code inside the 'if' block runs.

st.rerun() restarts the script from the top — which refreshes the page to show the newly selected page content.

How Page Navigation Works

Your app does not use multiple files for multiple pages. It uses one file with a simple if/elif chain. The active_page variable stored in session_state decides which section of code runs:

```
page = st.session_state.active_page

if page == 'Chat':
# ... all the chat code runs here

elif page == 'Statistics':
# ... statistics page code runs here

elif page == 'History':
# ... history page code runs here
```

```
    # and so on for each page
```

■ **Think of it like a TV remote.** There is only one TV screen (the app). When you press a channel button, the TV does not physically change — it just shows different content on the same screen. Your nav buttons change active_page, and the if/elif chain decides what content to display.

8

app.py — Part 3: Chat Logic

From Typing a Message to Seeing the Result

The Input Area

```
# Create two columns: wide one for text, narrow for button
col_input, col_send = st.columns([5, 1])

with col_input:
user_text = st.text_area("", placeholder="Type or paste...", height=80)

with col_send:
send_clicked = st.button("■ Send")
```

■■ **st.columns([5,1])** splits the screen into two side-by-side sections. The numbers [5,1] mean the first column is 5 times wider than the second. This creates the wide text box on the left and the narrow Send button on the right.

What Happens When Send Is Clicked

```
if send_clicked and user_text.strip():
# .strip() removes empty spaces from start and end
text = user_text.strip()

# Save the user's message to chat history
st.session_state.messages.append({"role": "user", "content": text, "time":
now})

# Run the AI classification
verdict, confidence, reasons = classify(text, model, vectorizer)

# Save the result to chat history
st.session_state.messages.append({"role": "result", "verdict": verdict, ...})

# Reload the page to display the new messages
st.rerun()
```

The classify() Function — The Heart of The System

```
def classify(text, model, vectorizer):
# Step 1: convert the new message into numbers using the saved vectorizer
```

```
    vec = vectorizer.transform([text])

    # Step 2: ask all 200 trees to vote
    pred = model.predict(vec)[0]

    # Step 3: get the probability — how many trees voted each way
    proba = model.predict_proba(vec)[0]
    phishing_conf = proba[1] # fraction that voted phishing
    legit_conf = proba[0] # fraction that voted safe

    # Step 4: decide the verdict based on confidence
    if pred == 1:
    verdict = "PHISHING" if phishing_conf >= 0.80 else "SUSPICIOUS"
    else:
    verdict = "SAFE" if legit_conf >= 0.75 else "SUSPICIOUS"

    # Step 5: run the pattern checklist for reasons
    reasons = analyze_reasons(text) if pred == 1 else []

    return verdict, confidence, reasons
```

■ **model.predict(vec)** — asks all 200 trees to vote. Returns 0 or 1.

model.predict_proba(vec) — returns the exact vote percentages. For example [0.08, 0.92] means 8% voted safe, 92% voted phishing — so the confidence score shown is 92%.

The SUSPICIOUS verdict kicks in when the model is not very confident (below 80% for phishing, below 75% for safe). This is the amber warning zone.

The analyze_reasons() Function

```
    def analyze_reasons(text):
    reasons = []
    text_lower = text.lower()

    # Check 1: Is there an IP address instead of a domain name?
    if re.search(r'http[s]?://(?:\d{1,3}\.){3}\d{1,3}', text):
    reasons.append("Contains IP address instead of domain name")

    # Check 2: Does it use a URL shortener?
    for p in SHORT_URL_PATTERNS:
```

```
    if p in text_lower:
    reasons.append(f"Uses URL shortener ({p})")

    # Check 3: urgent language?
    if any(k in text_lower for k in URGENT_KEYWORDS):
    reasons.append("Contains urgent pressure language")

    # ... more checks ...
    return reasons
```

■ **re.search()** uses something called a Regular Expression (regex) — a special pattern-matching language. The pattern **r'http[s]?://(?:\d{1,3}\.){3}\d{1,3}'** means: find 'http://' followed by three groups of 1-3 numbers separated by dots — which is exactly how an IP address looks (e.g. 192.168.1.1).

Displaying Chat Messages

The app loops through every message in st.session_state.messages and renders each one based on its role:

```
    for i, msg in enumerate(st.session_state.messages):
    if msg["role"] == "user":
    # show blue bubble on the right
    elif msg["role"] == "result":
    # show red/green/amber result card on the left
    elif msg["role"] == "feedback_request":
    # show Yes/No feedback buttons
    elif msg["role"] == "thanks":
    # show green thank you message
```

9

app.py — Part 4: Other Pages

Statistics, History, Settings, About

Statistics Page

```
elif page == 'Statistics':
# count_scans() reads the log CSV and counts totals
total_s, ph_s, sf_s = count_scans()

# load accuracy scores from the metrics.json file
acc = metrics.get('accuracy', 0)
prec = metrics.get('precision', 0)

# display 4 stat cards side by side using columns
c1, c2, c3, c4 = st.columns(4)
```

History Page

```
elif page == 'History':
# read the scan log CSV file
log_df = pd.read_csv(LOG_PATH, header=None,
names=['Timestamp','Message','Verdict','Confidence'])

# reverse it so newest scans show at the top
log_df = log_df.iloc[::-1].reset_index(drop=True)

# loop through each row and display as a colored card
for _, row in log_df.iterrows():
st.markdown(f'...{row["Verdict"]}...', unsafe_allow_html=True)
```

Settings Page — The Retraining Button

```
elif page == 'Settings':
if st.button('■ Retrain Model Now'):
with st.spinner('Retraining...'):
# run the train_model.py script as a separate process
import subprocess
result = subprocess.run(['python3', 'train_model.py'],
capture_output=True, text=True)
if result.returncode == 0:
```

```
st.success("Model retrained successfully!")
else:
st.error(f'Error: {result.stderr}')
```

■ **subprocess.run()** is how Python runs another Python script as if you typed it in the terminal yourself. When you click Retrain, it effectively runs 'python3 train_model.py' behind the scenes and waits for it to finish. If it succeeds, you see a green success message. If it fails, you see the error.

The save_feedback() and log_scan() Functions

```
# log_scan: writes every scan result to the history log
def log_scan(text, verdict, confidence):
with open(LOG_PATH, 'a', newline='') as f:
writer = csv.writer(f)
writer.writerow([datetime.now(), text[:80], verdict, confidence])

# save_feedback: saves a user correction to feedback.csv
def save_feedback(text, correct_label):
with open(FEEDBACK_PATH, 'a', newline='') as f:
writer = csv.writer(f)
writer.writerow([text, correct_label])
st.session_state.feedback_count += 1
```

■ **open(path, 'a')** opens the file in APPEND mode — meaning it adds a new line at the end without deleting what was already there. If the file does not exist yet, it creates it automatically. **'a'** = append. **'w'** = write (overwrites). **'r'** = read only.

10 The Full Data Journey

One Message — Start to Finish

Let us trace exactly what happens when someone types 'URGENT: Your PayPal is suspended. Verify at http://paypa1.tk' and clicks Send.

1	**Text arrives:** user_text = 'URGENT: Your PayPal is suspended. Verify at http://paypa1.tk' The .strip() removes any accidental spaces from start/end.
2	**Saved to messages:** st.session_state.messages.append({role:'user', content:text, time:'10:30 AM'}) The message is added to the chat history list.
3	**classify() is called:** classify(text, model, vectorizer) is called with the raw text.
4	**TF-IDF converts text:** vectorizer.transform(['URGENT: Your PayPal...']) converts the text into a row of 5000 numbers. 'urgent' gets score 0.72, 'verify' gets 0.68, 'paypal' gets 0.51. Most other positions are 0.
5	**200 trees vote:** model.predict(vec) sends the numbers to all 200 trees. 197 trees say phishing (1). 3 trees say safe (0).
6	**Confidence calculated:** model.predict_proba(vec) returns [0.015, 0.985] 98.5% of trees voted phishing → Confidence Score = 98.5%
7	**Verdict decided:** pred = 1 and phishing_conf = 0.985 which is >= 0.80 verdict = 'PHISHING'
8	**analyze_reasons() runs:** Checks: 'urgent' found ✓, 'verify' found ✓, 'paypa1' has suspicious domain ✓, no HTTPS ✓. Returns list of 4 reasons.
9	**Result saved to messages:** st.session_state.messages.append({role:'result', verdict:'PHISHING', confidence:98.5, reasons:[...]})
10	**st.rerun() fires:** Page refreshes. The loop renders all messages. User sees the red PHISHING card with 98.5% confidence bar and all 4 reasons listed.

11	**Log written:** log_scan() appends one row to logs/scan_log.csv: 2024-06-01 10:30:15, URGENT: Your PayPal..., PHISHING, 98.5
12	**Feedback shown:** feedback_request message is added. 'Yes correct' / 'No incorrect' buttons appear.

11 Common Errors & What They Mean

Reading Error Messages Like a Developer

Error messages look scary but they always tell you exactly what went wrong and where. Here is how to read them and what the most common ones mean:

Error: `ModuleNotFoundError: No module named 'streamlit'`

Fix: You have not installed the libraries yet. Run: pip install -r requirements.txt

Error: `FileNotFoundError: models/phishing_model.pkl`

Fix: You have not trained the model yet. Run: python train_model.py first.

Error: `FileNotFoundError: data/dataset.csv`

Fix: You have not generated the dataset yet. Run: python generate_dataset.py first.

Error: `StreamlitAPIException: set_page_config() can only be called once`

Fix: You have a st.set_page_config() call somewhere that is not the very first command in app.py. Move it to line 1.

Error: `ValueError: could not convert string to float`

Fix: The model is receiving text that was not properly converted by TF-IDF. Make sure you are using vectorizer.transform() not vectorizer.fit_transform() in the app.

Error: `Port 8501 is in use`

Fix: Streamlit is already running in another terminal. Either close it or run on a different port: streamlit run app.py --server.port 8502

Error: `The app stopped and nothing shows`

Fix: Check the terminal where you ran 'streamlit run app.py'. The real error message is always shown there, not in the browser.

Now you understand every line of your project — from the first import to the last st.rerun(). When a bug appears, you know where to look. When your supervisor asks how it works, you can explain it. You built this. You own it. ■